

0073A- Compilation Techniques

Lecturer: **Andrea Corradini**

andrea.corradini@unipi.it

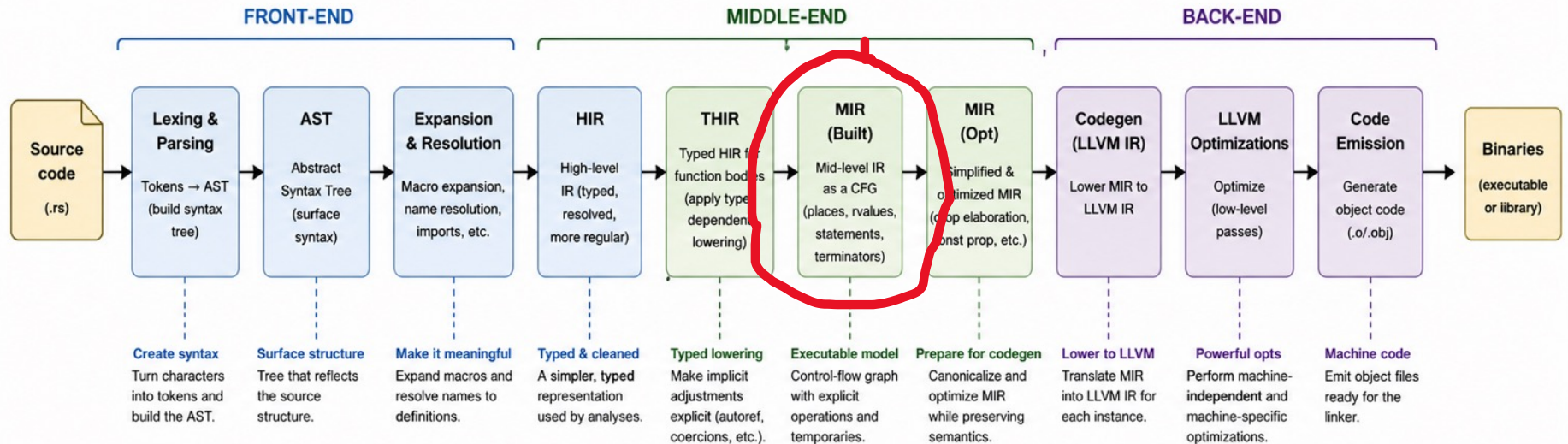
<http://pages.di.unipi.it/corradini/>

CompTech-26: RUST #3

Aspects of Rust Compilation (3)

- Borrow checking on the MIR
- Phases of borrow checker
- Some example
- Tracking moves and initialization
- Move analysis as Dataflow Analysis
- The MIR Type Check
- Region Inference for Non-Lexical Lifetimes
- Big Picture: Two Analyses, One Framework

The rustc Compilation Pipeline



● FRONT-END

- Understand the program text.
- Resolve names and types.
- Produce HIR for the crate.

● MIDDLE-END

- Build and transform MIR.
- Run borrow checking and other analyses.
- Prepare MIR for codegen.

● BACK-END

- Monomorphize generic code.
- Generate LLVM IR.
- Optimize and emit object code.

● AFTER rustc

- Link object files and libraries.
- Produce final binaries.

Key idea

Ownership and borrowing are enforced statically in the middle-end over MIR, well before code reaches the backend.

AST = Abstract Syntax Tree • HIR = High-Level IR • THIR = Typed HIR • MIR = Mid-Level IR • CFG = Control-Flow Graph

Ownership + Borrowing Rules

[O1] Every value is owned by a variable, identified by a name or path;

[O2] Each value has at most one owner at a time;

[O3] When the owner goes out-of-scope, the value is 'dropped'.

[B1] At most one mutable reference to a resource can exist at any time

[B2] If there is a mutable reference, no immutable references can exist

[B3] If there is no mutable reference, several immutable references to the same resource can exist

[B4] Owner cannot free or mutate its resource while it is immutably borrowed

[B5] Owner cannot even read its resource while it is mutably borrowed

Alias XOR Mutation

The MIR Borrow Checker

The Rust borrow checker is responsible for enforcing the Ownership and Borrowing rules, plus:

- *All variables are initialized before they are used.*

The borrow checker operates on the MIR

An older implementation operated on the HIR. Doing borrow checking on MIR has several advantages:

The MIR is *far* less complex than the HIR; the radical **desugaring** helps prevent bugs in the borrow checker.

[List of bugs that the MIR-based borrow checker fixes here.](#)

More importantly, using the MIR enables [“non-lexical lifetimes”](#), which are regions derived from the control-flow graph.

Borrow Checking as Program Analysis

- **Domain:** The checker reasons about **places**, **move paths**, **loans**, **initialized states**, and **regions** at locations in a MIR CFG.
- **Constraints:** Type checking over MIR generates **region constraints**, while **move** and **borrow analyses** produce facts over program points.
- **Dataflow:** Classic gen/kill and fixed-point ideas appear in the computation of **initialization**, **liveness**, and **loan validity**.
- **Diagnostics:** The result is not only accept or reject; the compiler maps facts back to source spans to explain why a borrow or move is invalid.

Major Phases of the MIR Borrow Checker

- **Entry point:** `mir_borrowck` query
The borrow checker is implemented in `rustc_borrowck` and is invoked as a query over MIR.
- **MIR duplication and preparation**
A local copy of MIR is created and will be **mutated in-place** to attach region information.
- **Region variable initialization**
`replace_regions_in_mir` replaces all lifetime annotations with **fresh inference variables**.
- **Dataflow analyses (moves and initialization)**
Analyses compute **when values are moved, initialized, or invalidated** across the CFG.
- **Second type checking over MIR**
A MIR-level type check derives **constraints between regions (lifetimes)**.
- **Region inference (constraint solving)**
Computes the **set of CFG points where each lifetime must hold**, via fixpoint-style inference.
- **Borrow sets / borrows in scope**
Determines which borrows are **active at each program point** in the control-flow graph.
- **Final validation pass (error reporting)**
A second traversal checks operations (e.g., `*a + 1`) against:
 - initialization state
 - borrowing rules
 - mutability constraints

A simple example

```
//Rust
fn example(a: &mut i32) -> i32 {
    let x = *a;
    *a = x + 1;
    x
}
```

```
//Simplified MIR
fn example(_1: &mut i32) -> i32 {
    let mut _0: i32;          // return
    let mut _2: i32;          // x

    bb0: {
        _2 = (*_1);           // read through borrow
        (*_1) = _2 + 1;        // write through borrow
        _0 = _2;
        return;
    }
}
```


Phase 1–2

Region Variables & MIR Preparation

- All lifetimes in MIR are replaced with fresh region variables
- Example transformation:

```
_1: &mut i32  →  _1: &'r1 mut i32
```

- These '**r1**', '**r2**', ... are unknowns to be solved later
- MIR is now annotated but not yet validated
- We move from explicit lifetimes → variables to infer

```
//Simplified MIR
fn example(_1: &'r1 mut i32) -> i32 {
    let mut _0: i32;           // return
    let mut _2: i32;           // x

    bb0: {
        _2 = (*_1);           // read through borrow
        (*_1) = _2 + 1;       // write through borrow
        _0 = _2;
        return;
    }
}
```

Phase 3

Dataflow (Moves, Initialization)

Compute:

- where variables are initialized
- where values are moved
- where they become invalid

Note: This is classical dataflow analysis on CFG (similar to liveness)

In our example:

- `_2` is initialized
 `at _2 = (*_1)`
 - `_1` is used but not moved
- No uninitialized reads

```
//Simplified MIR
fn example(_1: &'r1 mut i32) -> i32 {
    let mut _0: i32;           // return
    let mut _2: i32;           // x

    bb0: {
        _2 = (*_1);           // read through borrow
        (*_1) = _2 + 1;       // write through borrow
        _0 = _2;
        return;
    }
}
```

Phase 4–5 Constraints + Region Inference

Constraint generation

- From MIR operations:

`_2 = (*_1)` → requires:

'`r1` must be **valid at this program point**

`(*_1) = _2 + 1` → requires:

'`r1` still valid, no conflicting borrows

Region inference

- Solve constraints to compute:

`'r1 = {bb0 entry ... bb0 exit}`

- Interpretation:

→ lifetime must cover **all uses of `_1`**

- This is a **CFG-based, fixpoint-style analysis** (as in dataflow frameworks)

```
//Simplified MIR
fn example(_1: &'r1 mut i32) -> i32 {
    let mut _0: i32;          // return
    let mut _2: i32;          // x

    bb0: {
        _2 = (*_1);           // read through borrow
        (*_1) = _2 + 1;       // write through borrow
        _0 = _2;
        return;
    }}
}
```

Phases 6-7: Borrows in Scope + Final Check (Error Detection)

At each program point, compute:

IN[bb0]: { mut borrow of _1 }

- This tells us:
 - which references are **active**
 - which aliasing constraints must hold
- Conceptually similar to **live sets**, but for borrows

Now we validate each MIR statement:

Statement: **$(*_1) = _2 + 1$**

Check:

- Is $_1$ initialized? YES
- Is $_1$ mutably borrowed more than once? NO
- Any conflicting immutable borrows? NO
- ✓ OK

```
//Simplified MIR
fn example(_1: &'r1 mut i32) -> i32 {
    let mut _0: i32;
    let mut _2: i32;

    bb0: {
        _2 = (*_1);
        (*_1) = _2 + 1;
        _0 = _2;
        return;
    }
}
```

Another example

- Classic dangling reference (e.g. in C/C++)

```
fn bad() -> &i32 {  
    let x = 10;    // allocated on stack  
    &x  
}
```

```
//Simplified MIR  
fn bad() -> &i32 {  
    let mut _0: &i32;    // return place  
    let mut _1: i32;    // x  
  
    bb0: {  
        _1 = const 10;  
        _0 = &_1;    // borrow of local  
        return;  
    }  
}
```

Phase 1–2

Region Variables & MIR Preparation

- Lifetimes in MIR are replaced with fresh region variables
- Additional region variables for borrows:

```
_0: &i32    →  _0: &'r0 i32  
_1: i32  
_0 = &_1    →  _0 = &'r1 _1
```

```
//Simplified MIR  
fn bad() -> &i32 {  
    let mut _0: &'r0 i32;    // return place  
    let mut _1: i32;        // x  
  
    bb0: {  
        _1 = const 10;  
        _0 = &'r1 _1;        // borrow of local  
        return;  
    }  
}
```

Phase 4: Constraint Generation

From assignment: `_0 = &'r1 _1`

the following constraint is generated:

`'r1 : 'r0 (" 'r1 outlives 'r0")`

(borrow must live at least as long as return)

```
//Simplified MIR
fn bad() -> &i32 {
    let mut _0: &'r0 i32;    // return place
    let mut _1: i32;        // x

    bb0: {
        _1 = const 10;
        _0 = &'r1 _1;      // borrow of local
        return;
    }
}
```

Phase 5: Region Inference (CFG-based)

Compute where regions are valid:

- Program Points

```
bb0:  
  [p1] _1 = 10  
  [p2] _0 = &_1  
  [p3] return
```

- Inferred regions

```
'r1 = {p2,p3} // scope rule  
'r0 = {p3, caller context}
```

- Constraint Failure: **'r1: 'r0** requires all points in **'r0** to belong to **'r1**, which is false

```
//Simplified MIR  
fn bad() -> &i32 {  
  let mut _0: &'r0 i32; // return place  
  let mut _1: i32;      // x  
  
  bb0: {  
    _1 = const 10;  
    _0 = &'r1 _1;      // borrow of local  
    return;  
  }  
}
```

Note: The error is **not detected locally at the borrow**, but **globally after region inference**:

constraints + CFG + drops $\Rightarrow$ unsatisfiable

Tracking Moves and Initialization in the Borrow Checker

- **Unified view:** initialization = ownership state
The borrow checker tracks a set of “initialized places”;
 - assignment → adds to the set
 - move → removes from the set
- **Moves and initialization are the same analysis**
From the compiler’s perspective, both are just state transitions in dataflow.
- **Example behavior**
A variable becomes initialized after assignment, but a move (e.g., **drop(a)** or **b = a**) makes it uninitialized again, causing later use errors.
- **Granularity: "move paths", not variables**
Tracking is done at the level of move paths (e.g., **a**, **a.0**, **a.b.c**), enabling fine-grained reasoning about partial moves.

Tracking Moves and Initialization in the Borrow Checker

- **Unified view:** initialization = ownership state

The borrow checker tracks a set of “initialized places”;

- assignment → adds to the set
- move → removes from the set

- **Moves and initialization are the same**

From the compiler’s perspective, both `b = a` and `drop(a)` are moves. `drop(a)` or `b = a` makes it uninitialized.

- **Example behavior**

A variable becomes initialized after

`drop(a)` or `b = a` makes it uninitialized.

- **Granularity: “move paths”, not variables**

Tracking is done at the level of move paths (e.g., `a`, `a.0`, `a.b.c`), enabling fine-grained reasoning about partial moves.

```
fn foo() {  
    let a: Vec<u32>;  
  
    // a is not initialized yet  
  
    a = vec![22];  
  
    // a is initialized here  
  
    std::mem::drop(a); // a is moved here  
  
    // a is no longer initialized here  
  
    let l = a.len(); //~ ERROR  
}
```

Move paths

- **Move paths $\approx$ MIR Places (optimized)**

Each move path represents a memory location and is stored as a [MovePathIndex](#) for efficiency.

- **Construction phase**

Move paths are built by walking the MIR ([MoveData::gather_moves](#)), recording where each place is initialized and moved.

- **Illegal move paths are excluded**

No tracking is created for:

- array elements (`foo[1]`)
- dereferenced borrows (`*foo`)
→ reduces analysis overhead

- **Tree structure and lookup**

Move paths form a hierarchical tree (e.g., `a` $\rightarrow$ `a.b` $\rightarrow$ `a.b.c`), allowing queries on parents/children and efficient analysis of partial moves.

Move paths

- **Move paths $\approx$ MIR Places (optimized)**

Each move path represents a memory location and is stored as a [MovePathIndex](#) for efficiency.

- **Construction phase**

Move paths are built by walking the MIR ([MoveData::gather_moves](#)), recording where each place is initialized and moved.

- **Illegal move paths are excluded**

No tracking is created for:

- array elements (`foo[1]`)
- dereferenced borrows (`*foo`)
→ reduces analysis overhead

- **Tree structure and lookup**

Move paths form a hierarchical structure on parents/children and efficient lookup

```
fn foo() {  
    let a: (Vec<u32>, Vec<u32>) = (vec![22], vec![44]);  
  
    // a.0 and a.1 are both initialized  
  
    let b = a.0; // moves a.0  
  
    // a.0 is not initialized, but a.1 still is  
  
    let c = a.0; // ERROR  
    let d = a.1; // OK  
}
```

Move Analysis as Dataflow

- Initialization tracking as a **forward dataflow problem**:
- **Domain**
 - $IN[n], OUT[n] \subseteq \text{MovePaths}$
 - Each element = a **move path** (e.g., a , $a.0$, $a.b.c$)
- **Meaning**:
 - $IN[n]$: initialized before statement n
 - $OUT[n]$: initialized after statement n
- Standard forward dataflow equations:
- $OUT[n] = GEN[n] \cup (IN[n] - KILL[n])$
 $IN[n] = \bigcup OUT[p]$ for all predecessors p

Interpretation in Rust

- GEN[n] (initializations):

- assignments:

- a = ...**

- adds **a** to initialized set

- KILL[n] (moves):

- moves:

- let b = a;**

- drop(a) ;**

- removes **a** from initialized set

Example (Scalar case)

```
let a: Vec<u32>;  
a = vec![22];  
drop(a);  
let l = a.len(); // ERROR
```

Program Point	IN	GEN	KILL	OUT
entry	$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
a = ...	$\emptyset$	{a}	$\emptyset$	{a}
drop(a)	{a}	$\emptyset$	{a}	$\emptyset$
a.len()	$\emptyset$	—	—	ERROR

a $\notin$ IN $\rightarrow$ use of uninitialized value

Example: Field-Sensitive (Move Paths) (1)

```
let a: (Vec<u32>, Vec<u32>) = (vec![1,2,3], vec![4,5,6]);  
let b = a;
```

- Move paths: { a, a.0, a.1, b }

Statement	GEN	KILL
init a	{a, a.0, a.1}	∅
b = a	{b}	{a, a.0, a.1}

- Result:
 - Initialized = {b}
- Moving a parent kills all children

Example: Field-Sensitive (Move Paths) (2)

```
let a: (Vec<u32>, Vec<u32>) = (vec![1,2,3], vec![4,5,6]);  
let b = a.0;
```

- Move paths: { a, a.0, a.1, b }

Statement	GEN	KILL
init a	{a, a.0, a.1}	∅
b = a.0	{b}	{a.0}

- Result:
 - Initialized = {a, a.1, b}
 - a.0 is dead, but a.1 is still valid
- Moving a child does NOT kill siblings

Example: Field-Sensitive (Move Paths) (3)

```
let a: (Vec<u32>, Vec<u32>) = (vec![1,2,3], vec![4,5,6]);  
let b = a.0;  
let c = a;
```

- Move paths: { a, a.0, a.1, b, c }

Statement	IN	GEN	KILL
init a	∅	{a, a.0, a.1}	∅
b = a.0	{a, a.0, a.1}	{b}	{a.0}
c = a	{a, a.1, b}	{c}	{a, a.0, a.1}

- Error:
 - use of partially moved value **a**

The MIR type check

- A key component of the borrow check is the [MIR type-check](#). This check walks the MIR and does a complete “type check”.
- During this type-check, it also uncover [region constraints](#) that apply to the program replaces all regions in the body [with new unconstrained regions](#).
- Even though MIR is built from THIR—which is already fully type checked—rustc still performs a MIR type check because [the lowering process is not type-preserving](#) in a trivial way and introduces new constructs and invariants that must be validated independently.

The MIR type check (2)

- During the transition from THIR to MIR, the compiler performs **desugaring**, **introduces temporaries**, **reorders evaluation**, **inserts implicit operations** (like borrows, moves, and drops), and **simplifies expressions** into a control-flow graph.
- These transformations can expose inconsistencies or require additional guarantees that were not explicitly checked at the THIR level.
- The MIR type check ensures that all these lowered operations are internally consistent, that operands and places have the correct types, and that invariants required by later phases (borrow checking, optimizations, codegen) hold.
- In other words, it acts as a defensive validation step: THIR guarantees the correctness of the source-level program, while MIR type checking guarantees the correctness of the compiler's own transformation into MIR.

Regions and lifetimes

- In rustc terminology, a **region** is the compiler's internal representation of a **lifetime**, used during type checking and borrow checking.
- A region denotes a **set of program points (or scope in the CFG)** where a reference is considered valid. Instead of thinking of lifetimes as purely syntactic scopes, the compiler models them as regions that can be **compared**, **constrained** (e.g., one region **must outlive another**), and **inferred** through dataflow-like analysis.
- The relationship is that **“lifetime” is the user-facing concept**, while **“region” is the compiler's semantic and analytical model** of it.
- When you write lifetime annotations like `'a`, the compiler translates them into region variables and constraints (e.g., `'a : 'b` becomes “region `'a` outlives region `'b`”).
- With non-lexical lifetimes (NLL), these regions are no longer tied to lexical blocks but correspond to precise sets of CFG points, computed via **constraint solving**.
- So, lifetimes in the language are essentially **names or abstractions over regions**, while regions are the concrete objects manipulated by the borrow checker.

Region Inference (NLL) — Overview

- Two main phases
 - `replace_regions_in_mir`: prepares MIR for inference
 - `compute_regions`: solves region (lifetime) variables
- Step 1: Identify universal regions
 - Extract free regions from function signature (e.g., `'a`, `'static`)
 - Represent caller-visible lifetimes
- Step 2: Replace regions with variables
 - All MIR regions → fresh inference variables
 - Discards lexical lifetime info → enables control-flow-sensitive (NLL) analysis
- Step 3: Generate constraints (MIR type check)
 - Specialized MIR type checker produces:
 - outlives constraints (`'a`: `'b`)
 - other relationships between regions
- Step 4: Solve constraints (region inference)
 - Build `RegionInferenceContext`
 - Compute region values via constraint propagation / fixpoint solving

Region Representation

- Regions are **sets of program points**
 - Each region = set of **CFG locations** where it is valid
- Region elements include
 - **MIR locations** (program points)
 - **end('a')** → lifetime extends into caller
 - **end('static')** → rest of program
 - placeholders for unknown regions
- Efficient implementation
 - Stored as bitsets indexed by **RegionElementIndex**

Constraints Driving Inference

- Outlives constraints
 - `'a: 'b` $\rightarrow$ `'a` must include everything in `'b`
 - Generated by [MIR type checker](#)
- Liveness constraints
 - [Region must be valid wherever it is used](#)
- More about Constraint Propagation later

Region Inference (NLL) — How It Works

- Initialization from liveness
 - Each region starts with CFG points where it is used
 - These come from liveness constraints
- Constraint propagation
 - For each outlives constraint 'a : 'b :
$$'a \leftarrow 'a \cup 'b \cup \{\text{end}('b)\}$$
 - Repeated until fixpoint ([propagate_constraints](#))
- Regions grow monotonically
 - Region values are sets of elements
 - Propagation = union-based dataflow

Error Checking After Inference

- Type tests
 - Check constraints like: **T: 'a**
 - Ensures types respect lifetime requirements
- Universal region check (“too big” regions)
 - If 'a contains **end('b)** $\Rightarrow$ must already know 'a: 'b
 - Otherwise $\rightarrow$ error

```
fn f<'a, 'b>(x: &'a i32) -> &'b i32 {  
    x  
}
```

// NO

```
fn f<'a, 'b>(x: &'a i32) -> &'b i32  
where  
    'a: 'b,  
    { x }
```

// OK

- Constraints lead to: **'a contains end('b)**
- But no declared bound 'a: 'b, thus ERROR
- Region inference allows regions to grow via constraints, but **growth must be justified by known relationships.**

RegionInferenceContext (Core Data)

- Holds all inference inputs:
 - constraints → outlives relations
 - liveness_constraints → initial seeds
 - universal_regions → free lifetimes
 - universal_region_relations → known bounds (e.g. where clauses)
 - type_tests → post-check constraints
- Solving process:
 - solve():
 - propagate_constraints
 - check_type_tests
 - check_universal_regions

Big Picture - Two Analyses, One Framework

- Rust borrow checking = two interacting dataflow analyses on MIR
 1. Move / Initialization Analysis (ownership)
 2. Region Inference (NLL) (lifetimes)
- Both are:
 - CFG-based
 - monotone
 - solved via **fixpoint iteration**

Shared Dataflow Structure

Aspect	Move Analysis	Region Inference (NLL)
Domain	Set of move paths	Set of region elements
Direction	Forward	Forward (propagation)
Lattice	Powerset	Powerset
Transfer	GEN/KILL	Union via constraints
Meaning	“is initialized?”	“is region valid here?”
Fixpoint	Yes	Yes

- Equations:
 - $OUT[n] = GEN[n] \cup (IN[n] - KILL[n])$
 $IN[n] = \bigcup OUT[p]$
 - GEN = assignments
 - KILL = moves
 - $R_a = LIVENESS_a \cup \bigcup \{ R_b \cup end(b) \mid a: b \}$
 - Start from liveness seeds
 - Propagate via outlives constraints

Differences and Interaction

- Move analysis: can remove information (kills)
- Region inference: only adds information (grows sets)
- Where They Interact: Combined reasoning
 - Move analysis: $\text{drop}(x) \Rightarrow x$ not initialized
 - Region inference: r must be valid at use
 - Conflict:
 - region says “valid”
 - move analysis says “value gone”

```
let x = vec![1];  
let r = &x;  
drop(x);  
*r; // error
```